

The Orchestrated Coding Team

How I build working software with a team of AI agents without being a developer. An experience report, plus a blueprint you can copy.

Fatih Altıok · 5 October 2026

Adapted from the German original, which is published alongside this version. I wrote this together with Claude, the AI model that leads the team as its orchestrator. That's a bias, and I come back to it under "Limits".

Version 1.3, 5 October 2026. Every empirical number in this paper is listed, with the file it comes from and a script that checks it, in the public companion repository github.com/fatihaltiok/orchestrated-coding-team, fixed release v1.3 (file numbers.csv; what can be recomputed and what comes only from the project log is set out under "Data and reproducibility"). Version 1.3 completes the list of sources, adds that section, recounts the memory and corrects one sentence about leaks. Version 1.2 (5 October 2026) tied every number to a file; version 1.1 (1 October 2026) extended "Related work" after a second literature search; version 1.0: [doi:10.5281/zenodo.23086547](https://doi.org/10.5281/zenodo.23086547).

The short version

I don't write code, and honestly I can't really read it either. Still, I run about 25 projects at the same time: tools for clients, tools for my own work, films, course material. The code comes from AI agents. My part is to say what should exist, to approve anything that costs money or goes public, and to set the rules.

What makes this work is a team with a lead, not one clever agent. One model, Claude, is the orchestrator. It plans, writes down how the parts have to fit together, hands out the work and checks every result. Several coding agents from different companies build the parts, each one in its own copy of the project.

Everything rests on one idea: **if it matters, it lives in a file and a tool checks it. Nobody has to remember it.** AI models forget. When their working memory fills up, they summarize the conversation so far, and a 2026 study planted standing instructions such as "do not delete any emails until I confirm" in test sessions and found that the summarizers it tested kept only 17 % of them on average ([Wang et al. 2026](#)). So every instruction I give, every decision and every task is a file, and wherever we can, a small program checks the rule instead of hoping someone remembers it.

Since the first version of this report (19 September 2026), five things were added:

- A shared memory of lessons learned, 209 notes on 5 October 2026, with an automatic check for dead links and orphaned entries.
- The "context package". Before an agent starts, it gets all my rules word for word and every decision that's still valid. A checksum file lets a tool notice if anything is missing or out of date.
- One rule file for all agents, generated for each tool. Before that, the copy Codex was reading had quietly lost 31 of 137 rules.
- A small judging model, Jev, that sorts extra reading material by relevance. In our test it put 34 relevant sections into the top ten. Plain keyword search got 20.
- A team channel between Claude and Codex. Two lead sessions now message each other directly, and a message wakes up an idle Claude window without me typing anything. On top of that: a handover when a session runs out, a list of questions for me, and a "done" message after every agent run.

Most of the building blocks aren't new. What I haven't found anywhere else is this combination, run by someone who never looks at the code. That's what this report is about. It ends with a blueprint (Appendix A) so others can set up something similar.

Where I'm coming from

Not being a developer gives me three problems a single agent can't solve:

- I can't catch bugs myself. So I need checks I can trust more than my own eyes.
- My instructions have to stick. Something like "ask me before you spend money" holds for weeks, across dozens of sessions and restarts.
- Big projects have to stay manageable. The bigger a project gets, the more a single agent would have to keep in its head, and the more it forgets.

The answer looks a lot like a normal human team: a lead who plans and checks, specialists who build, and written agreements that don't depend on anyone's memory. The one big difference: everyone on this team except me forgets everything the moment their session ends. So the whole setup has to expect forgetting from day one.

How it grew

None of this was planned on a whiteboard. Each stage came out of something that went wrong.

Stage	Since	What went wrong	What we do now
1 One agent	before Aug 2026	Bigger programs filled up the agent's memory, instructions got lost when it summarized, and whoever builds a bug tends to miss it again when checking.	—
2 A team in one folder	Aug 2026	The borders between agents were just agreements. A checking agent once undid a fix that had just been made. Tests ran against half-finished work from several agents at once. Background runs died without a word.	An orchestrator plus specialist agents, with the work split into areas. Since 18 Sept a small tool, the run watcher, starts every run and only counts it as finished if a "done" file exists.
3 Own working copies	19 Sept 2026	Agreements don't stop accidents.	Every agent works in its own copy of the project. Nothing gets merged unless the diff has been saved and shown and the tests pass in exactly that state.
4 A checked memory	21 Sept 2026	Our collected lessons had no rules at all. In ten places, notes pointed to files that didn't exist anymore, and we believed them anyway.	Written rules for the memory and a tool that checks its shape on every run.
5 Context package, one rule file, judging model	26–27 Sept 2026	An agent broke a build rule it had never been shown. The rule lists in big projects were longer than any agent reliably reads. Codex's rule file was out of date.	Every task gets a context package that can be checked; all agents' rules come from one file; Jev sorts the extra material.

Stage 5 started with a message from me on 26 September. It said, roughly (translated from German): this should work across all projects, "so we can work more efficiently, faster, safer and smarter, no matter how complex the projects get." It was built in eight packages (U0 to U7) over two days, by the team's own agents, under the same rules they were putting in place.

The team

The agents come from different companies on purpose. Every model has blind spots, and a mistake one of them misses, another one often catches. Not always, though. Strong models from different vendors often make the same mistakes ([Kim et al. 2025](#)), and in one study a review by another vendor's model lowered the share of passing solutions instead of raising it ([Xiang et al. 2026](#)). That's why our reviewer never changes anything, and a finding only counts once the orchestrator has reproduced it. The main staffing rule: **whoever reviews never comes from the same company as whoever built it.**

Role	Who (as of 27 Sept 2026)	What for	What we've seen
Orchestrator	Claude (Anthropic)	Planning, writing contracts, handing out tasks, checking every result, putting things together	Writes hardly any code itself; it's the quality gate
Builder	Codex (OpenAI)	Core code, measuring tools	Thorough, but the usage allowance runs out fast
Builder, small jobs	Devin CLI (Cognition)	Tightly scoped tasks with named files	On the team since 27 Sept: six runs in three packages (eight in four by the end of that day); two runs stopped, correctly, because their context package was out of date
Reviewer	Muse Spark (Meta)	Reviewing other models' code, small areas	Finds real gaps; sometimes quits on big jobs or right before the final report

Role	Who (as of 27 Sept 2026)	What for	What we've seen
Reviewer (paused)	Kimi K3 (Moonshot)	Second opinion, very large contexts	Gave the most thorough reviews (up to 28 findings); out of credit right now
Builder (paused)	GLM 5.3 (Zhipu)	Was the workhorse until 27 Sept	Flat rate, huge context; subscription cancelled on 27 Sept
Backup	Mercury (Inception)	Small areas	Very cheap; ignored a build rule in its trial run
Checker	Sonnet (Anthropic)	Restart checks, blind judging in measurements	Fresh eyes with no background knowledge
Judge	Jev (TypeSafe)	Yes/no calls, rankings, matching	Writes no code; one judgment costs a fraction of a cent

The line-up keeps changing. In two weeks two models dropped out (one ran out of credit, one subscription got cancelled) and two joined. So the process hangs on roles and rules, not on any particular model. When the line-up changes, one paragraph in the rule file changes. The process stays the same.

Here's how the "other company reviews" rule played out during the upgrade on 26 and 27 September:

Package	Built by	Reviewed by	Findings
U7 Memory into the context package	Devin	Muse	2, both real (a third was only noted for later)
U6 Small fixes	Devin	Kimi	0
U5 Jev in the context package	Codex, Devin	Kimi	8, one of medium weight
U3 One rule file	GLM	Kimi, Muse	7 and 3
U2 Jev module	Muse	Codex	6, including 3 leaks: two through which the access key could have gone out, one of a full file path
U1 Context package	GLM	Codex	12

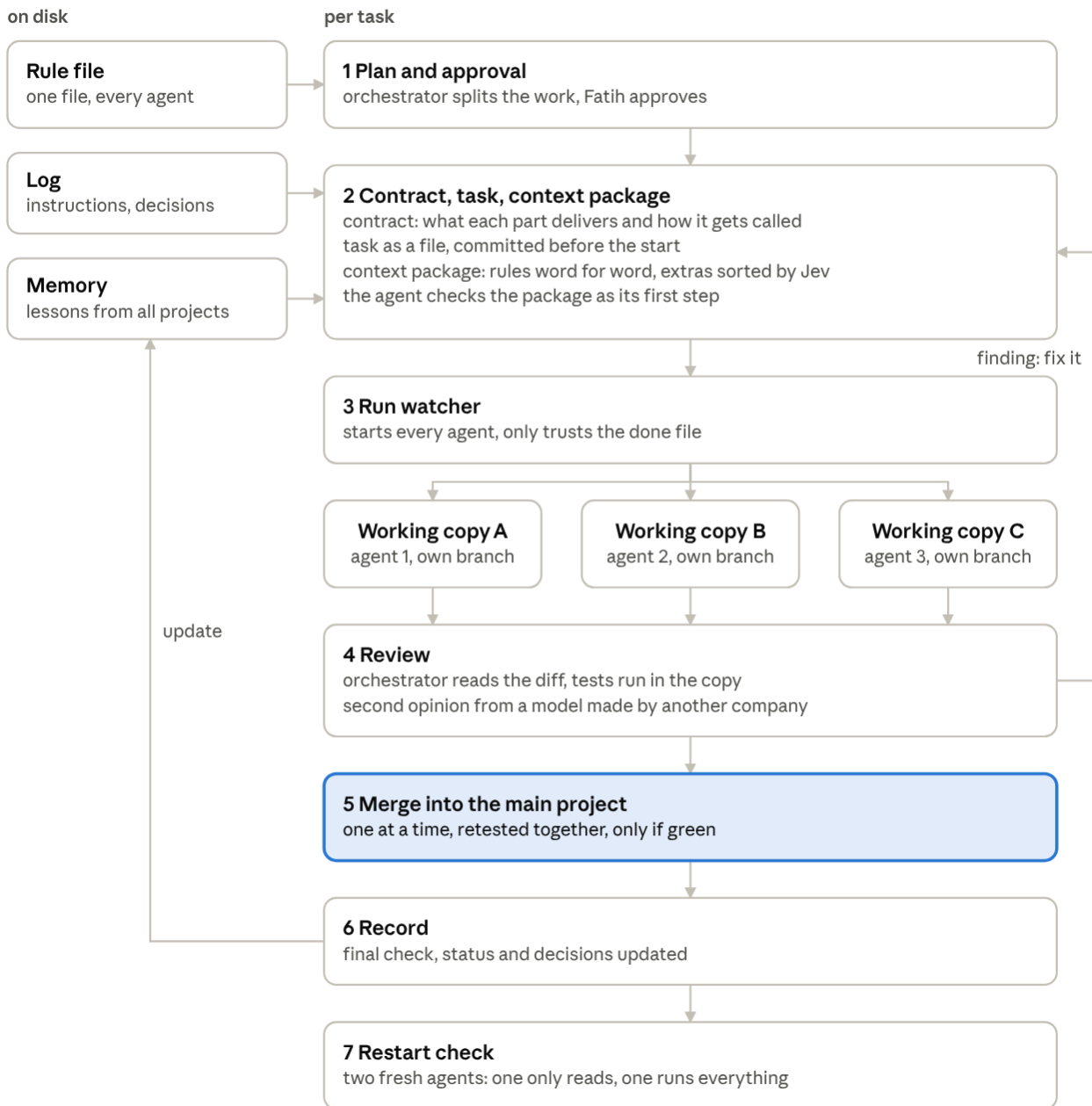
Package	Built by	Reviewed by	Findings
UO Contracts	Claude	Kimi	28, all taken on board

The architecture

There are two sides. On the left is what lives on disk permanently and survives every restart. On the right is the path every single task takes. Each step leaves a file behind. There's only one way back: if the review finds something, the task gets reworked and built again.

Nothing reaches the main project unless it was shown and checked

Left: what lives on disk for good. Right: the path every task takes



Architecture · three things kept on disk, seven steps per task

The building blocks:

Block	What it does	Tool
Rule file	One file with all working rules. The rule files for Claude and for Codex are generated from it, and a check reports if someone edited a generated file by hand.	<code>regeIn</code> (generate, install, check)

Block	What it does	Tool
Log	Every instruction I give, word for word, and every decision with the rejected alternative and the reason. Things only get added, never cut.	<code>PROGRESS.md</code> per project
Memory	Lessons from all projects, as an index plus single notes.	Memory folder, checked by <code>merkzettel-check</code>
Contract	Says before anything is built what each part delivers and how it's called. Without it, parts built in parallel don't fit.	<code>INTERFACES.md</code> or <code>VERTRAG.md</code> per project
Task	Exact instructions per agent: area, rules, acceptance tests, and a last step "create the done file".	A file under <code>docs/auftraege/</code>
Context package	All my current rules word for word, every valid decision with where to find it, the current status, plus extra material in the order Jev suggests. A checksum file makes every package verifiable.	<code>kontextpaket bau / pruefe</code>
Run watcher	Starts each agent as a supervised process in its own working copy and reports done, running, aborted or stuck. It doesn't go by what the agent says: a run without the done file counts as aborted, whatever the agent reports. Whether the result is right is decided later, in review.	<code>laufwaechter start / warte</code>
Review	The full diff is saved and shown; without that step the tool refuses to merge. Actually reading it is the orchestrator's job, no tool can force that. Tests run in the copy.	<code>laufwaechter ergebnis</code>
Merge	Pulls the latest main project into the copy, tests everything together, and only merges if it's green. Rejected work stays on its branch.	<code>laufwaechter abschliessen</code>

Block	What it does	Tool
Record	Reports anything uncommitted, not backed up, forgotten working copies, and rule files that drifted from their source.	<code>abschluss-check</code>
Restart check	Two fresh agents test whether the handover notes are enough for a restart. One only reads, the other really runs the commands.	Agents with no background knowledge

All tools live in one shared repository and every project uses them through short commands. There are deliberately no per-project copies, because copies drift apart. The tools themselves are covered by 499 automated tests (counted on 27 September 2026). (The tool names are German, because that's the language I work in.)

One real example, step by step

On 27 September 2026 we wanted the team's memory to go into every context package (package U7). From my OK to "done" took less than an hour. Here's what happened:

1. **I decide.** I wanted the collected lessons in every task: "I need your notepad in the folder, we don't always have client data" (translated). The orchestrator had advised against it, because some notes contain client names. The decision went into the log with both sides: what was chosen, what was rejected, and why.
2. **A plan in plain language.** The orchestrator told me who builds (Devin), who reviews (Muse), and what it costs (review about 1–2 dollars, Jev under 2 cents per package). My answer: "yes, start".
3. **Contract first.** The orchestrator wrote the contract section itself: what behavior is required, how to switch it off, and which eleven test cases have to prove it.
4. **Task and context package.** Both were written as files and committed before the agent started. What isn't committed doesn't make it into the working copy.
5. **Start in its own copy.** The run watcher started Devin. Its first move was to check the context package: is it complete and current? An out-of-date package stops the run. That actually happened earlier the same day on another package, and stopping was the right call.
6. **Look, then merge.** Once the done file was there, the tool saved the state, the orchestrator read the diff against the contract, and the tests ran in the copy. Only then was it merged.

7. **Review by another company.** Muse read the code without changing anything and found two real gaps. A dead link in the notes folder made every package build fail, and a link pointing outside the folder got read too, so outside files could have ended up in the package and been sent on to Jev.
8. **Reproduce, then fix the contract.** The orchestrator reproduced both findings itself before treating them as real, and added to the contract. The hole was in the contract; the builder had just followed it.
9. **Fix.** Devin fixed it in a fresh working copy. Afterwards Muse's own evidence scripts no longer found anything, and all 499 tests passed (27 September 2026).
10. **Real trial.** A context package for a real task showed the right notes, Jev cost about half a cent, and the memory stayed untouched.
11. **Wrap-up.** Final check, status updated, restart check by two fresh agents. It was only pushed after my explicit OK ("yes do it").

On bigger projects, several tasks like this run at the same time, each in its own copy. They're merged **one after another**, and each merge first pulls in the newest main project and tests how everything works together. The part that connects everything comes last, on its own. For critical pieces there's the **double build**: two agents from different companies build the same task independently, the orchestrator keeps the better result and throws the other away.

Protecting context

What "context" means

Every AI model has a limited working memory, its **context**. Everything it reads, writes or gets back in a session takes up space. When it's full, the conversation so far gets **summarized**. What gets lost is whatever seems unimportant to the summary: details, reasons, and above all instructions like "ask me first". The study "Lost in Compaction" (2026) measured that such instructions survive summarizing only 17 % of the time on average. The rate varied with the summarizing model, its prompt and the wording of the instruction, but no setting fixed the loss. The authors show a way out: a small extra model runs alongside, collects the instructions in its own list and adds them back after summarizing, which keeps over 90 %. We follow the same idea with different means: the instructions sit word for word in files, and tools check that they reach the agent. We haven't tested our approach on their benchmark.

Why a team helps

- The orchestrator doesn't read piles of files. The agents read the code; what comes back is a report and a diff. Its context stays free for architecture, review and the big picture.
- Each agent only needs its own area. With its own copy, a clear task and a context package, an agent doesn't have to understand the whole project. Models with smaller memory only get small, tightly scoped tasks.
- Waiting costs nothing. The run watcher waits up to nine minutes at a time and then reports back, instead of the orchestrator asking every few seconds.

The context package

"Read all the project's rules and decisions before you start" sounds simple. But on 26 September, in our biggest project, those lists were about 305,000 bytes long, roughly 76,000 tokens. No agent reads that reliably from start to finish, and nobody can see what it skipped. That's exactly how an agent had ignored a build rule two weeks earlier.

The context package fixes this by splitting things in two:

- **The must-read part, complete and word for word:** every instruction I've given that still applies, every valid decision at least with its core sentence and where to find it, the decisions relevant to this task in full, and the current status. No model is allowed to remove anything from this part.
- **The extra part, only sorted:** lessons from the memory and related sections. Jev suggests the reading order. Nothing gets dropped.

Every package comes with a **checksum file** listing all sources. The tool `kontextpaket` `pruefe` uses it to notice when an instruction is missing, a source has changed since the package was built, or the status is stale. If a package gets too big, it's **split into parts, never cut off**. We rejected two tempting shortcuts: having a model summarize the lists (it would reword my instructions, which is the very thing we're trying to prevent) and "only the newest entries" (old decisions that still apply would silently drop out).

One rule file for every agent

The working rules sit in a file every model loads at the start of each session and again after every summary. But Claude and Codex read different files, and we used to maintain them by hand. A comparison on 26 September found that the Codex copy was **missing 31 of 137 rules, and 7 more said something different**. It also mentioned a tool that had been gone for days.

Since 27 September there's **one source**, and both files are generated from it. When we switched, the generated Claude file had to match the old one byte for byte, so we knew for sure no rule got lost. Since then, the final check reports any installed file that drifts from its source. A quick test showed the difference: afterwards Codex named the right second-opinion model, the right command for fix-up rounds, and the tool rules it had been missing.

We did the same with the project index every session loads. It had grown to 76,983 bytes, because each new status got written above the old one. Codex only read the first 32,768 bytes of it, so 43 %. Now it's 16,542 bytes: per project just the entry file and one status line that replaces the old one. The history lives in the projects. A small script keeps size and format in check.

How knowledge survives a restart

The main principle: **if a rule can be checked by a program, it becomes a check, not a sentence.** A check that fails doesn't need anyone to remember anything. Our layers of protection, strongest first:

Rank	Layer	Why it holds
1	Things that run: tests, gates, tool locks, package and rule checks	They run fresh every time; no model has to remember
2	Decision log	Keeps the <i>why</i> that isn't in the code, including what we rejected
3	Rule file	Gets reloaded after every summary
4	Instruction list	My instructions word for word, never summarized
5	Status as free text	The first thing to go stale

Across all five sits the **memory** (next section), the only layer that works across projects.

The upgrade moved a lot of things up that list. "Don't touch other people's files" used to be an agreement between agents, and now the working copies keep them physically apart. "Check before merging" is now a lock in the tool. And "read all the rules" turned into a package with a checksum.

Two habits back up the tools:

- Tasks are files. An agent never gets just one sentence in a command, it gets a task file in the repository. If the context is lost, the task is still there.

- The restart check: before every restart and after every finished package, two fresh agents with no background knowledge go through the handover notes. One only reads and lists every spot where it would have to guess. The other actually runs the commands and reports what doesn't match. Both read the original files, never just the context package. Otherwise one mistake in the selection would hide the same gap from both, and them agreeing would just be an echo.

The memory

The team's memory is a folder with a short index and **209 single notes** (counted on 5 October 2026). Most of them are lessons from mistakes: which model gets what wrong, which check doesn't actually measure anything, which command has a trap in it. The index-plus-notes setup comes with Claude Code: the index is loaded in every session without asking, and notes are read when needed. What we added is a form check, a lossless restructuring instead of trimming, and putting the memory into the context package. It's the only layer that works across projects. Something learned on a film project is there when a client project needs it.

A wrong note gets believed

Until 21 September 2026, this biggest store of knowledge had **not a single written rule**, and nothing checked it. That's worse than a missing test in code: a broken test turns red, a wrong note gets believed. The first run of our new checker, `merkzettel-check`, found ten places where notes pointed to files that no longer existed, plus five links between notes that led nowhere. One of those notes described a film that had been lost in a data loss on 20 September as if it were still there.

Since then, the **shape** of the memory is rank 1. Nothing is orphaned, no link points nowhere, the index stays under its loading limit, and a path written in code format is a promise that it exists. When something's gone, the note keeps it, but without the code format and with a date. The **content** stays manual work. No tool can tell if a note is still true, and the contract says so on purpose, so a green run doesn't get mistaken for a healthy memory.

No vector database

The obvious move would have been a vector database that finds notes by meaning instead of words. Our own test on the real notes said otherwise. For three search questions, plain `grep` found the right note in the top four twice, the best meaning-based model only once. The study "Is Grep All You Need?" (2026) reaches a similar result in

some of its comparisons. It looked at conversation memory, not code, though, and stresses that the result depends a lot on the agent tool. Three questions are a hint, not proof, and we can change our minds.

The deeper reason is something else. A database improves *how well* things are found, not *whether* anyone looks. A model doesn't look something up if it has no idea there's something to know. What helps is a short index that's loaded without asking, not a better search engine.

Keeping details instead of trimming

When the index hit its loading limit, the easy fix would have been to merge similar notes. I said no: "no, that's not good, if you have to leave out details, we'll do it differently" (translated). Keeping details is the whole point of the memory; cutting it down to fit a limit turns that upside down. We solved it without losing anything: several links per index line, and project statuses that had crept into the index moved into the notes, after a script had checked that every sentence had made it over. 165 index lines became 45, and no note was shortened.

Jev, a judging model

Jev (made by TypeSafe) isn't a language model that writes text or code. It answers narrow questions with a number or a choice, fast and for a fraction of a cent. On our team it has a tightly limited role, written into the contract:

- Jev sorts and never filters. It suggests in what order extra material should be read. It never takes anything out of the must-read part of a context package.
- Jev approves nothing. No judgment from Jev replaces a test, a review or a decision by me.
- If Jev fails, you see it. Then plain keyword order applies, and the package says "not rated".
- Every request has a control: a text with nothing to do with the task goes along and has to get a low score. If it doesn't, the model isn't judging, it's just nodding.
- Nothing secret goes out. Before sending, a tool looks for patterns of passwords and access keys and holds those sections back. The model version is fixed, so a silent model update can't spoil the measurement.

First use: ask about the action, not a number

On 21 September we wanted to know if the memory had duplicate notes. A similarity score didn't help: the values for all pairs were spread out evenly, with no clear gap between "similar" and "different". Any cut-off would have been arbitrary. Jev's own

documentation suggests something else: **ask directly about the action**, with three levels matching the three things you can do: keep them apart, show them to a human, merge them. Then there's no threshold to tune to your data.

The full run over all 3,541 note pairs took 40 seconds. Result: not a single duplicate, but 137 related pairs that didn't link to each other yet. The judgment was 98 % repeatable. The one pair that looked like a duplicate the first time flipped on the second run, because it sat exactly on the border between two levels. **Near a border, you read it yourself, you don't round.** And the strongest matches had almost no words in common. A keyword pre-filter would have missed all of them.

Measuring before rollout

Before Jev was allowed into every context package, we measured it on 27 September. The setup was fixed and committed beforehand: five real tasks, a reference judgment from two blind Sonnet judges per task (so also language models, not people; they agreed 93 % of the time, and the orchestrator decided the ten disputed cases), a decision rule with five conditions, and a spending cap of 10 cents. The reference was in the repository before Jev gave a single answer.

Measure (five tasks together)	Keyword search	Jev
relevant sections in the top 5	5	18
relevant sections in the top 10	20	34
control passed	—	10 of 10 runs
same judgment on a repeat run	—	98.7 %
cost and time per task	\$0, instant	about \$0.003, about 9 s

All five conditions were met. I made the call, and since 27 September Jev sorts in every project. But the test also showed a limit: **55 of 94 irrelevant sections still got a high score.** So Jev's scores are useless for filtering things out. Only the ranking is worth anything. The test confirmed our "sort, never filter" rule, which had only been a precaution before. That fits the research on re-ranking search results: single scores from such models make poor cut-offs, the ranking works well. The vendor talks about "calibrated probabilities". But what they measure is agreement with large models, not relevance against a reference like ours, and they say themselves that thresholds have to be set per use case.

Data protection

Jev runs on the vendor's servers, so task texts and notes leave our hands. Reading the privacy policy word for word: TypeSafe promises in writing not to train on the data, but keeps it with no fixed deletion date in the US. Zero retention is only for big customers. So we switched Jev off for the one project with confidential company data. For the others I accepted the transfer on purpose. The decision is in the log with both sides, and it can be reversed per project with one line.

Quality checks

An agent saying "done" has only made a claim. Code is done when it passes objective checks, the orchestrator has read it, and an outside reviewer finds nothing more. The chain:

1. **Spec before building.** The contract says what has to be tested before any agent writes code.
2. **Gates with a counter-test.** Automated tests, and stricter methods for critical code. Every test has to fail when you put the bug back in on purpose. Otherwise it's not testing anything.
3. **Review by the orchestrator.** It reads the code against the contract, not just the test results.
4. **Review by another company.** Read-only, with evidence for every finding. The orchestrator reproduces each finding before it counts, which also weeds out the reviewers' false alarms.
5. **A real-world run.** The result gets used once on real material, not just tested.

What actually happened

In **every** bigger package since 19 September, the building agent reported green tests. In almost every one, a later step found real bugs anyway:

Date	What was built	Builder's tests	Found anyway	By
27 Sept	Memory in the context package	green	A dead link made every package build fail; a link pointing outside got read too	Muse

Date	What was built	Builder's tests	Found anyway	By
27 Sept	Jev in the context package	green	A made-up example key in some text switched Jev off for the whole task; real key formats from other services weren't recognized	real-world run; Kimi
26 Sept	Jev module	green	Three leaks through which an access key or a full file path could have gone out	Codex
26 Sept	Context package	green	12 findings; one rule change put 21 decisions in the wrong category, only visible when comparing counts per project	Codex; real-world run
21 Sept	Memory checker	117 of 117	Two kinds of false alarm, and a warning that flagged 63 of 154 notes, more than anyone would ever go through	first run on the real notes
19 Sept	Working-copy mode	67 of 67	Six gaps (three from the outside review, four from my own, one of them found by both), one of which would have let someone else's unreviewed work into the main project	Muse; orchestrator

The restart check keeps finding real problems too. On 19 September it caught a table that listed a test gate as green when it was red. On 27 September the two checkers read a line about the upload status in exactly opposite ways. That line now names the exact commit.

What we learned

- 1. Green tests show that what was tested is right, not that the right thing was tested.**
The worst bugs were found by outside eyes and real material, not by tests.
- 2. Every new check gets run once against the real thing.** Test cases only contain what somebody thought of.

3. **Thresholds get tuned on real data.** A sharper word list flagged 16 notes instead of 63. Same check, useful instead of useless.
4. **The gap is usually in the contract, not in the builder's work.** So we fix the contract first and then rebuild, never the other way round.
5. **Whoever builds can't sign off their own work.** That goes for the agents and for the orchestrator: its own contracts go to an outside reviewer too.

The team channel

Since 1 October 2026, two lead sessions, Claude and Codex, can work together as equal colleagues. Before that I was the messenger: I copied text from one window into the other.

The team channel is a small mailbox on my own computer. Every session gets its own ID when it starts. A message to Claude shows up there as a new turn, even if nobody is typing. A message to Codex goes into the running session through Codex's own command. Stored, delivered and read are three different states: a message only counts as read once the receiver confirms it.

What we've seen in real use

- **30 Sept:** In the first live test, messages went both ways, for \$0.04; only the last read confirmation was missing. Then the test got stuck because Codex hit its usage limit. Codex's small quota has been the bottleneck ever since.
- **1 Oct, early:** A message from Codex woke up an idle Claude window without me typing anything. The first try wasn't clear, because a command arrived in the window at the same moment. The second one was.
- **1 Oct, afternoon:** A small test run, a command that only waits 5 seconds, reported "done" by itself right after it ended, in the Claude session that had started it.

Three features, picked with Jev

Which uses are worth it? Claude wrote down 13 possible scenarios, and Jev rated each one on time saved, how often it happens, mistakes avoided, and whether something else already covers it. The rating cost 0.06 cents. I picked the top three, and the team built them the same day. The script and its results are in the team's repository.

Feature	Jev score (0-1)	What it does
Handover when a session runs out	0.81	A session that's running out of context or quota leaves its status: done, half done, next step, open approvals. The handover belongs to the project, not to a session, and can be taken over exactly once.
Question list for me	0.79	Questions only I can decide go on a list. I answer in the terminal or in any window, and the answer wakes up the session that asked.
"Done" message	0.78	The run watcher tells the session that started an agent run when the run ends. In team windows this has been required since 1 Oct.

Last place went to a scenario we deliberately don't use: letting the colleague session check the handover notes (0.41). It already knows the status, so it isn't independent. The team channel doesn't replace fresh, blind reviewers.

Conversations without me starting them, but with limits

At first only I could open a conversation. On 1 Oct I decided: "make them do it automatically, without a switch". Since then the sessions open conversations themselves. Some limits are enforced by the program, some are rules:

Limit	Enforced by the program?
At most 10 conversations between sessions per project and day	yes
Only with exactly one session that's currently running, otherwise the question goes on my list	shown, not enforced
At most six messages per conversation	default and rule; the program enforces the limit set for each conversation, which can go up to 20
Colleague messages, handovers and answers from the question list are never an approval	rule

The daily limit wasn't my idea. The orchestrator suggested it, so that two sessions can't keep waking each other up in a loop and burn through the Codex quota.

How it was built

The features were made like everything else: first the contract, then two build tasks in parallel, each in its own working copy. Codex with GPT-6.1 Sol built the team channel part, MiMo Flash the run watcher part. Reviews went crosswise, each from a different vendor: MiMo Pro checked the Codex part, Devin the MiMo part. Neither found anything blocking, but each found two gaps. For example, the answer to a very long question could never fit into one message. And a "done" message didn't arrive if a run was repeated under the same name. Three of the four gaps were fixed. One was left open on purpose and is only described: if the program dies exactly between saving an answer and handing it to Codex, the hand-off isn't repeated, so nothing arrives twice. After that, 752 tests passed (1 October 2026).

For me it's one command: `team projectname` opens a window with the team channel for that project. Then I just talk as usual.

What the team channel can't do

- **It doesn't start a new session.** If no other session is running, the handover waits until I'm back. That's on purpose: no program should use up quota without me noticing.
- **A sudden limit prevents the handover.** If the Codex plan cuts off abruptly, the session can't write anything anymore. So before I leave, I say: "write a handover."
- **Codex isn't woken by handovers**, only Claude is. Codex sees them only when it checks itself.

- **Two lead sessions can change the same file.** Their agents have working copies, but for the sessions themselves there's only the rule "one area, one owner". On 1 Oct two Claude sessions worked in the same status file on the same day, and I had to decide who was in charge. So my advice is: one leads and builds, the other advises and reviews.
- **Handover and question list haven't been tried in real use yet**, only in automated tests with real processes. After two or three real uses I'll judge whether they're worth it.

The human's role

The human on the team doesn't have to read code, but stays in control. That works because the process only puts in front of me what I can actually judge, and leaves the rest to reviewers who do read code.

- I decide the *what*, not the *how*. Instead of "framework A or B?" the orchestrator asks "should it run in the browser? should data be kept permanently?". It does the technical translation itself and explains its recommendation in one sentence.
- No agent starts without a plan and my OK. The plan comes in plain language: who builds, who reviews, what it costs, what's different afterwards.
- An OK is good for one thing only. Anything that costs money, goes public (uploading, emails, publishing) or changes the shared rules needs an explicit OK, every single time. An OK to build a tool isn't an OK to install it. Every approval goes into the log word for word, with the date.
- I always know where things stand. Before and after every bigger step, the orchestrator tells me without being asked who's doing what, what's done, what's in progress and what's still open.

The orchestrator names the risks, I decide. A few times I was right where it had advised caution:

- "The full run with Jev is cheap anyway": the orchestrator only wanted to check the 200 most promising note pairs. The full run found the strongest connections exactly where the pre-filter wouldn't have looked.
- "No, that's not good, if you have to leave out details": instead of merging notes, we built a solution that loses nothing.
- "I need your notepad in the folder": the orchestrator had advised against it because of client names. Now every agent gets the collected lessons, and the key scanner plus the per-project off switch keep the limits.

The other way round, the orchestrator never quietly takes a different path from what was decided. Anyone who wants to change a valid decision first has to name the existing entry and give a reason, and I have to agree.

Related work

Almost every building block here has been done before. What's new is mostly the combination and the role of the human. That's the result of a search on 27 September 2026 for which about 100 sources were opened and read, and of a second search on 1 October 2026 with the literature agents of Edison Scientific, whose hits we looked up one by one. Two hits we couldn't find anywhere are left out. "Not found" only means not found in what we searched. We didn't go through the proceedings of the big software engineering conferences systematically.

Block	Already out there	Not found like this
Orchestrator, one working copy per agent, merge after tests	A planner that fixes the interfaces and hands out work to specialist agents (Hong et al. 2023). One working copy per agent, tests as a gate and agents from several vendors are normal practice in 2026 (Osmani 2026); tools like Gas Town test every change on the merged state. A central manager agent that hands subtasks to agents in isolated git worktrees and merges after tests beat a single agent by 25.6 and 14.7 points on two benchmarks (Geng & Neubig 2026). Agent claims like "tested" or "done" count only with fresh evidence tied to the current code state (Huang et al. 2026).	A tool lock that refuses to merge until the diff has been saved; testing the merged state locally in the copy.

Block	Already out there	Not found like this
Review by another company, double build	Models favor their own output (Panickssery et al. 2024) and hardly correct themselves without outside feedback (Huang et al. 2023). The double build is good old N-version programming (Knight & Leveson 1986); with coding agents too, shared failures pile up where the spec is ambiguous (Ron et al. 2026). A build-and-review loop split between two vendors, with a requirements model as the contract, ran on two production systems (Zahran 2026).	A fixed staffing rule across a changing team; a read-only reviewer, every finding reproduced by the orchestrator.
Instructions and decisions	An extra model against losing instructions (Wang et al. 2026); measuring whether withdrawn instructions keep acting in multi-turn chats (Zhu 2026); decision logs as Architecture Decision Records (Nygard 2011 , MADR).	Instructions word for word in a file only the human can revoke; a required "checkable by" field.
Context package	Stale agent context as a failing CI check (Braun 2026). Forcing agents to check remembered constraints works, hints don't (Nakayashiki 2026). Instructions fade in long agent runs; one remedy is reminders the system injects at fixed events (Bui 2026).	A package with checksums that the agent checks itself as its first move; split, never cut.
Memory	Index plus single notes is Claude Code's built-in memory (Claude Code docs).	An automatic form check of that memory.

Block	Already out there	Not found like this
One rule file	Ready-made tools that generate rule files from one source and report drift (Ruler , rulesync); the shared standard AGENTS.md .	Byte-for-byte sign-off when switching over; a number for how many rules got lost between two tools (here 31 of 137).
Judging model	Re-ranking results with language models (Sun et al. 2023); models use the start and end of a text best, so order matters (Liu et al. 2024). The "sort" recipe and asking about the action come from Jev's own docs.	A measurement fixed in advance and independent of the vendor, before rollout.
Restart check	Progress files for long agent runs (Young 2025); documentation that gets run like a test (Docs as Tests).	Two independent fresh checkers, one only reads, one runs things, both on the original files.
Non-developer in charge	Non-programmers can't reliably check AI-generated code (Virk & Liu 2025 ; Fawzy et al. 2026). Professionals stay in control by steering the code themselves (Huang et al. 2025). Approval gates before actions with consequences (Mozannar et al. 2025). A non-programmer who built with Claude Code and Cursor documents a failure costing over \$1,000: 86 added subsystems contributed less than 2 % to the model's output (Aditto 2026).	A person who never reads code and steers the team through one-time approvals logged word for word, over weeks and many projects.

So here's what this report can offer and what it can't. It doesn't describe a big new mechanism. It describes a combination, a few small tool rules we haven't seen elsewhere, and field data that, as far as we can tell, nobody has published yet: which check found which bug, how many rules got lost between two tools, and how a judging model was measured before rollout. Above all it shows the flip side of the existing studies: someone

with no coding skills staying in control, because checking is handed over completely to tools and outside reviewers. The second search found closer relatives (a manager agent with isolated worktrees, gates that accept only fresh evidence, a build-and-review loop split between two vendors), but no report of a non-developer steering such a team. That holds as of 1 October 2026. It doesn't show that this way of working beats others.

Data and reproducibility

Every number in this report is listed in the public companion repository (release v1.3): `numbers.csv` names the source file for each of the 103, and one command, `make check`, compares each number with that file. 39 of them a script computes: 38 from the raw data, which `make results` writes word for word to the folder `results/`, and one, the spending cap, from a setting of the measurement tool. 46 stand word for word in a data file, and 18 come only from the dated project log, because their raw data was not kept. `make test` runs the 782 tests of the published tools.

Measurement	What is public	Recomputed by
Judging model before rollout (27 Sept 2026)	task list, blind reference judgments, every raw run	<code>make results</code> → <code>results/u4-jev-ranking.csv</code>
Duplicate run over the memory (21 Sept 2026)	all 3,541 rated note pairs, note names replaced by neutral ids	<code>make results</code> → <code>results/dedup-run.csv</code>
Team channel scenarios (1 Oct 2026)	Jev's scores, token count, list price	<code>make results</code> → <code>results/team-channel-ratings.csv</code>
Rule files of two tools (31 of 137 lost)	the rule-by-rule comparison	<code>make results</code> → <code>results/rule-file-comparison.csv</code>
Reviews and agent runs	one line per finding and per run	<code>make results</code> → <code>results/review-findings.csv</code> , <code>review-gaps.csv</code> , <code>devin-runs.csv</code>
Memory size (5 Oct 2026)	a snapshot with one entry per note, no names, no content	<code>make results</code> → <code>results/memory-notes.csv</code> (counted from <code>data/memory-snapshot/notes-2026-10-05.json</code>)

Recorded in the dated project log only. These numbers were written down at the time, but their raw data was not kept, so they cannot be recomputed: the ten dead file paths and five dead note links of the first memory check, and its 63 and later 16 flagged notes (21 Sept 2026); the three search questions comparing `grep` with meaning-based search (grep found the right note in the top four twice, the best meaning-based model once); the 40 seconds and the 98 % repeat rate of the first duplicate run; the 21 decisions put in the wrong category by one rule change; the six working-copy gaps (four from my own review, one found by both); the two Claude sessions on one status file; the one project with Jev switched off; the one pair that flipped; the two kinds of false alarm. Each of them sits in the repository as a dated excerpt from the log (folder `data/recorded/`), marked as such in `numbers.csv`.

Not public. The private repositories of the projects (client work, a company project, my instructions word for word) stay private. The tools that run the process are public, with their tests, so the process itself can be run and checked.

Limits

- **This is experience, not a study.** The measurements are small (three search questions, five tasks), and there's no comparison group that worked without this setup. The numbers show where the process caught bugs, not how many it prevents overall.
- **Tried by one person, not a team of people.** Whether it holds up when several people give approvals is an open question.
- **More admin.** Every run has a branch, a copy, a context package, a result, a merge and often a review. That's on purpose, because it swaps trust for proof, but it takes time.
- **Depending on vendors.** In two weeks two models dropped out. The "other company reviews" rule always needs at least two available vendors, and several vendors means several bills.
- **The orchestrator makes mistakes too.** Once it forgot the context package, once it built a package before it had finished editing the docs. Both times a check caught it, and since then the run watcher warns when a package is missing. Only what's checked by a program can be relied on, and that goes for the orchestrator as well.
- **The memory's content can't be checked.** No tool can say if a note is still true.
- **Data leaves our hands.** The coding agents and Jev run at vendors, some in the US. For client data and confidential company data that has to be decided per project.
- **Git only.** The working copies need version control. Media projects without Git, and runs that only read or report, go without them.

- **Badly split areas lead to conflicts.** The working copy prevents damage, but not two agents changing the same spot. Then nothing gets merged and the orchestrator has to redo it. Good planning is still half the battle.
- **False "stuck" alarms.** If an agent works outside its folder for a while or writes one huge file in one go, the run watcher wrongly reports it as stuck. Then someone looks at the session log.
- **There's evidence against parts of this.** Working copies alone just push conflicts to merge time ([Liu et al. 2026](#)). Changes from different agents clash twice as often as changes from the same agent, 41.7 % vs 19.8 % ([Xu et al. 2026](#)). Context files like AGENTS.md don't generally raise success rates and cost over 20 % more ([Gloaguen et al. 2026](#)). Strong models from different vendors often make the same mistakes ([Kim et al. 2025](#)), and a review by another vendor's model can even lower the pass rate, depending on who reviews whom ([Xiang et al. 2026](#)). Our precautions (split areas, contracts, merging one at a time, read-only reviewers) respond to that, but we haven't measured whether they help.
- **The team channel is new.** It has been running since 1 October 2026. Delivery and waking up are proven. Handover and question list have only been tested. Whether two lead sessions work better together than one hasn't been measured.
- **Bias.** Large parts of this report were drafted with the orchestrator (Claude, by Anthropic), which means a participant is describing its own way of working. I am the sole author and responsible for all of its content.

What's next

A big program doesn't come out of one long agent run for us. It comes out of **a series of small, finished packages**, each one small enough to plan, build, test and check on its own:

1. **Skeleton and contracts first.** Modules, data flows and interfaces are fixed before any agent writes code.
2. **Areas in parallel.** Independent parts run at the same time in their own copies, each with the agent that suits it.
3. **Critical stuff twice.** Core rules, money, data, security: two agents from different companies, the orchestrator picks.
4. **Merge early and often.** Every merge tests how things work with everything that's already there. So there's no big risky assembly at the end.
5. **Every phase ends with something that runs**, and it gets checked before the next one starts.
6. **Every important rule becomes a check.** If it can be checked by a program, it becomes a test or a lock instead of a sentence in the docs.

The upgrade on 26 and 27 September was built that way itself: eight packages, each with a contract, a working copy, a review and a restart check.

Still open: By 10 October 2026 we'll decide if Devin stays once its free period ends. The other projects get their context package settings the next time we work on them, not in one big sweep. And after a few weeks of use we want to look at whether admin work, false alarms or conflicts are slowing things down noticeably. The same goes for the team channel: after two or three real uses we'll check whether handover, question list and "done" messages actually save work, using Jev's rating from 1 October as the yardstick.

Appendix A: A blueprint to copy

This is the minimum you need to set up an orchestrated team yourself, without knowing our tools. You can bring in the blocks one at a time, in this order; each one helps on its own. For each there's a way to check that it actually works. A block without that check is just a good intention.

No.	Block	Minimum version	Does it work?
1	One rule file	All working rules in one file; the version each agent tool reads is generated from it, never edited by hand.	Every agent answers a test question whose answer is only at the end of the file. That tells you the file isn't being cut off, not whether the rules are followed.
2	Log	Per project, a list of the human's instructions, word for word with dates, and a list of decisions with the rejected alternative and the reason. Only add, never cut.	A fresh agent can explain <i>why</i> something is built this way and not another.
3	Tasks as files	Every task is a file in the repository with area, rules, acceptance tests and the last step "create the done file".	A run without the done file counts as aborted, whatever the agent says.
4	Supervised start	Every agent runs as its own supervised process; it's judged only by the done file and the process state.	A run you kill on purpose shows up as aborted, not as done.

No.	Block	Minimum version	Does it work?
5	A working copy per run	Every run that writes code gets its own copy on its own branch (in Git: <code>git worktree</code>).	The agent can't change the main project; rejected work stays on its branch.
6	Merge with a lock	Only merged after the diff has been saved and shown (the orchestrator has to read it) and the tests pass in the copy on the newest state.	A test you break on purpose blocks the merge.
7	Outside review	Critical packages get reviewed by a model from another company, read-only, with evidence per finding. The orchestrator reproduces every finding.	A bug you plant on purpose gets found.
8	Context package	Before every task, a script collects all current instructions word for word, the valid decisions and the status, with checksums. The agent checks the package as its first step.	Deleting an instruction makes the check fail.
9	Final check	Before calling anything done: nothing uncommitted, nothing not backed up, no forgotten working copies, no links to temporary folders.	A file left lying around gets reported.
10	Restart check	Two fresh agents check the handover independently: one only reads, one really runs the commands. Both read the originals.	They find spots where they'd have to guess, and say so.

Example task file

```
# Task U7-1: <title in one sentence>
```

```
You are the coding agent that DOES the work, not the orchestrator.
You have approval. Don't ask back.
```

```
## First step
```

```

Check the context package docs/auftraege/U7-1-kontext.md.
If the check reports anything: report it, stop working.

## Your area (only change these files)
- werkzeuge/kontextpaket.py
- werkzeuge/tests/test_kontextpaket.py

## Contract
VERTRAG.md §16 – binding. If something contradicts it: report, don't guess.

## Acceptance
- all tests green: python3 -m pytest werkzeuge/tests -q
- every new test fails if you put the bug back in

## Last step
Create the file docs/auftraege/U7-1-FERTIG.
Don't stop before that file exists.

```

Example decision entry

```

### E-022 | 2026-09-27 | valid
- Decision: <what was decided, one sentence>
- Rejected alternative: <what else would have been possible>
- Reason: <why the alternative lost, with a measurement if there was one>
- Affected paths: <files, modules>
- Checkable by: <test or command – or "can't be checked by a program">

```

The format follows Architecture Decision Records (Nygard 2011). What we added: "checkable by" is required, and the whole log goes into every agent's task. The last field matters most. If you ask for every decision whether it can be turned into a check, you slowly move it from "has to be remembered" to "gets checked every time".

Ten rules to take away

1. If it matters, it lives on disk, not in a model's head.
2. Instructions are kept word for word, never summarized.
3. Decisions get written down with what was rejected and why, right away, not at the end.
4. Contract first, then code.
5. Every agent that writes code works in its own copy.
6. "Done" is a claim; checks decide if it's true, not the report.
7. Nobody reviews their own work; review comes from another company.
8. Every check gets run once on real material.

9. A judging model sorts. It doesn't filter and it doesn't approve.
10. Costs, anything public, and rule changes get approved by the human, every single time.

Appendix B: A few terms

Term	Meaning here
Orchestrator	The lead of the team: plans, hands out work, checks, puts things together
Agent	An AI model that does a task on its own, here mostly writing or reviewing code
Area	The part of a project one agent is responsible for
Contract	The written agreement on what a part delivers and how parts talk to each other
Working copy	A separate copy of the project for one agent, on its own branch (<code>git worktree</code>)
Done file	A file the agent creates as its very last step; only that means "done"
Gate	A check that has to pass before things move on
Counter-test	Putting a bug back in on purpose to see if the check catches it
Context	A model's working memory during one session
Summarizing (compaction)	What happens when the context is full; details and instructions get lost
Context package	Everything an agent must know before starting, with checksums
Checksum	A short number calculated from a file's content. Change one character and the number changes, so a tool can tell right away if a source is still the same
Log	The list of the human's instructions (word for word) and of decisions (with reasons)
Restart check	Two fresh agents testing whether the handover is enough to restart

Sources

Every entry below was looked up on its original page on 5 October 2026. Research papers first, then practice sources and documentation; both in alphabetical order.

Research

- Aditto, A. (2026). Eyla: Toward an Identity-Anchored LLM Architecture with Integrated Biological Priors – Vision, Implementation Attempt, and Lessons from AI-Assisted Development. arXiv:2604.00009. [doi:10.48550/arXiv.2604.00009](https://doi.org/10.48550/arXiv.2604.00009)
- Bui, N. D. Q. (2026). Building Effective AI Coding Agents for the Terminal: Scaffolding, Harness, Context Engineering, and Lessons Learned. arXiv:2603.05344. [doi:10.48550/arXiv.2603.05344](https://doi.org/10.48550/arXiv.2603.05344)
- Fawzy, A., Tahir, A., Blincoe, K. (2026). From Prompting to Verification: How Experience Shapes Vibe Coding Practices. arXiv:2605.24521. [doi:10.48550/arXiv.2605.24521](https://doi.org/10.48550/arXiv.2605.24521)
- Geng, J., Neubig, G. (2026). Effective Strategies for Asynchronous Software Engineering Agents. arXiv:2603.21489. [doi:10.48550/arXiv.2603.21489](https://doi.org/10.48550/arXiv.2603.21489)
- Gloaguen, T., Mündler-Sasahara, N., Müller, M. N., Raychev, V., Vechev, M. (2026). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv:2602.11988. [doi:10.48550/arXiv.2602.11988](https://doi.org/10.48550/arXiv.2602.11988)
- Hong, S., Zhuge, M., Chen, J., et al. (2023). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352. [doi:10.48550/arXiv.2308.00352](https://doi.org/10.48550/arXiv.2308.00352)
- Huang, Jek, Hsia, J., Sun, J., Shi, F., Huang, W., White, I. H. (2026). Proof-or-Stop: Don't Trust the Agent, Trust the Evidence – Loop Engineering for Verifiable Evidence-Gated Lifecycle Control. arXiv:2607.14890. [doi:10.48550/arXiv.2607.14890](https://doi.org/10.48550/arXiv.2607.14890)
- Huang, Jie, Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., Zhou, D. (2023). Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798; ICLR 2024. [doi:10.48550/arXiv.2310.01798](https://doi.org/10.48550/arXiv.2310.01798)
- Huang, R., Reyna, A., Lerner, S., Xia, H., Hempel, B. (2025). Professional Software Developers Don't Vibe, They Control: AI Agent Use for Coding in 2025. arXiv:2512.14012. [doi:10.48550/arXiv.2512.14012](https://doi.org/10.48550/arXiv.2512.14012)
- Kim, E. M., Garg, A., Peng, K., Garg, N. (2025). Correlated Errors in Large Language Models. Proceedings of ICML 2025, PMLR 267:30038–30066. proceedings.mlr.press/v267/kim25e.html
- Knight, J. C., Leveson, N. G. (1986). An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. IEEE Transactions on Software Engineering SE-12(1):96–109. [doi:10.1109/TSE.1986.6312924](https://doi.org/10.1109/TSE.1986.6312924)
- Kopp, O., Armbruster, A., Zimmermann, O. (2018). Markdown Architectural Decision Records: Format and Tool Support. 10th Central European Workshop on Services and their Composition (ZEUS 2018), CEUR Workshop Proceedings Vol. 2072, 55–62. [ceur-](https://ceur-ws.org/Vol-2072/)

[ws.org/Vol-2072](https://adr.github.io/madr). Cited as "MADR"; current format: adr.github.io/madr

- Liu, M., Chen, T., Xu, Z., Jiang, X., Dong, Y. (2026). Multi-agent Collaboration with State Management. arXiv:2605.20563. [doi:10.48550/arXiv.2605.20563](https://doi.org/10.48550/arXiv.2605.20563)
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. Transactions of the ACL 12:157–173. [doi:10.1162/tacl_a_00638](https://doi.org/10.1162/tacl_a_00638), aclanthology.org/2024.tacl-1.9
- Mozannar, H., Bansal, G., Tan, C., et al. (2025). Magentic-UI: Towards Human-in-the-loop Agentic Systems. arXiv:2507.22358. [doi:10.48550/arXiv.2507.22358](https://doi.org/10.48550/arXiv.2507.22358)
- Nakayashiki, K. (2026). When Stale Constraints Go Unchecked: Budgeted Verification Failures in Inherited Agent Memory. arXiv:2608.25553. [doi:10.48550/arXiv.2608.25553](https://doi.org/10.48550/arXiv.2608.25553)
- Panickssery, A., Bowman, S. R., Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations. arXiv:2404.13076. [doi:10.48550/arXiv.2404.13076](https://doi.org/10.48550/arXiv.2404.13076)
- Ron, J., Baudry, B., Monperrus, M. (2026). N-Version Programming with Coding Agents. arXiv:2606.20158. [doi:10.48550/arXiv.2606.20158](https://doi.org/10.48550/arXiv.2606.20158)
- Sen, S., Kasturi, A., Lumer, E., Gulati, A., Subbiah, V. K. (2026). Is Grep All You Need? How Agent Harnesses Reshape Agentic Search. arXiv:2605.15184. [doi:10.48550/arXiv.2605.15184](https://doi.org/10.48550/arXiv.2605.15184)
- Sun, W., Yan, L., Ma, X., Wang, S., Ren, P., Chen, Z., Yin, D., Ren, Z. (2023). Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. Proceedings of EMNLP 2023, 14918–14937. [doi:10.18653/v1/2023.emnlp-main.923](https://doi.org/10.18653/v1/2023.emnlp-main.923)
- Virk, Y., Liu, D. (2025). Non-programmers Assessing AI-Generated Code: A Case Study of Business Users Analyzing Data. IEEE VL/HCC 2025; arXiv:2508.06484. [doi:10.48550/arXiv.2508.06484](https://doi.org/10.48550/arXiv.2508.06484)
- Wang, Z., Zhang, Y., Lee, D., Yang, Y. (2026). Lost in Compaction: Evaluating Side-Constraint Loss under Context Compaction. arXiv:2608.11242. [doi:10.48550/arXiv.2608.11242](https://doi.org/10.48550/arXiv.2608.11242)
- Xiang, Z., Zhang, Y., Zhang, Y., Xu, H. (2026). Cross-Model LLM Code Review: Should you use Claude to review Codex or vice versa? Agentic SE @ KDD 2026; arXiv:2607.21656. [doi:10.48550/arXiv.2607.21656](https://doi.org/10.48550/arXiv.2607.21656)
- Xu, G., Subramanian, A., Karthik, N. (2026). AI Agent Pull Requests on GitHub: Frequency, Structure, and Merge Conflict Rates. arXiv:2607.04697. [doi:10.48550/arXiv.2607.04697](https://doi.org/10.48550/arXiv.2607.04697)
- Zahran, M. (2026). From Requirements to Production: Governing AI-Assisted Software Delivery through a Canonical Requirements Model. Research Square preprint, 21 July 2026. [doi:10.21203/rs.3.rs-10400858/v1](https://doi.org/10.21203/rs.3.rs-10400858/v1)
- Zhu, H. (2026). Dead text or binding clause? Measuring and restoring constraint influence in black-box LLM dialogues. arXiv:2608.12599. [doi:10.48550/arXiv.2608.12599](https://doi.org/10.48550/arXiv.2608.12599)

Practice sources and documentation

- AGENTS.md: a shared format for instructions to coding agents. [agents.md](#)
- Anthropic. How Claude remembers your project (Claude Code documentation; cited as "Claude Code docs"). code.claude.com/docs/en/memory
- Braun, E. (2026). I made stale coding-agent context fail CI instead of failing silently. DEV Community, 6 August 2026. dev.to/agentskit/i-made-stale-coding-agent-context-fail-ci-instead-of-failing-silently-3434
- Gas Town: multi-agent orchestration for coding agents. GitHub repository. github.com/gastownhall/gastown
- Nygard, M. (2011). Documenting Architecture Decisions. Cognitect blog, 15 November 2011. cognitect.com/blog/2011/11/15/documenting-architecture-decisions
- Osmani, A. (2026). The Code Agent Orchestra – what makes multi-agent coding work. Blog post, 26 March 2026. addyosmani.com/blog/code-agent-orchestra
- Ruler: apply the same rules to all coding agents. GitHub repository intellectronica/ruler. github.com/intellectronica/ruler
- rulesync: rule files for AI development tools from one source. GitHub repository dyoshikawa/rulesync. github.com/dyoshikawa/rulesync
- Silva, M. Docs as Tests. docsastests.com
- TypeSafe AI. Documentation of Jev, cookbooks "Re-ranking" and "Entity Alignment"; read 26 September 2026. docs.typesafe.ai, docs.typesafe.ai/cookbooks/rerank_typesafe.md, docs.typesafe.ai/cookbooks/entity_alignment.md
- TypeSafe AI. Privacy Policy (last updated 19 November 2025) and Data processing addendum (last updated 24 April 2026); both read in full 27 September 2026. typesafe.ai/privacy-policy, typesafe.ai/legal/data-processing
- Young, J. (2025). Effective harnesses for long-running agents. Anthropic, 26 November 2025. anthropic.com/engineering/effective-harnesses-for-long-running-agents

All other numbers come from our own measurements and logs (decisions E-001 to E-028, the contract, the Jev measurement report of 27 September 2026, Jev's rating of the team channel scenarios of 1 October 2026); which of them can be recomputed from the public companion repository and which come only from the dated project log is set out under "Data and reproducibility". The process and the tools were built in 2026 by me working with Claude as orchestrator, and this report was written the same way. Quotes from me are translated from German. The second literature search (1 October 2026) used the literature agents of Edison Scientific; every hit cited here was looked up on its original page.